Estudi previ P-PRE3

MQTT, Mosquitto al Raspberry Pi i primers fluxos de Node-RED

Pau Vila Pérez

Bloc A · El protocol

A1. Per què MQTT fa servir un intermediari

Amb MQTT els dispositius no es parlen directament: tots es connecten al broker. Qui té dades les publica en un tema, i el broker les fa arribar a tots els que estan subscrits a aquell tema. Qui publica no sap qui l'escolta. Tres conseqüències:

- Desacoblament: per afegir un consumidor nou (un dashboard, una base de dades) no cal tocar el node. Només s'ha de subscriure al tema.
- El node només obre una connexió, cap al broker. No ha de fer de servidor ni saber l'adreça de ningú.
- Inconvenient: el broker és un punt únic de fallada. Si el Raspberry s'atura, no arriba res a ningú, encara que el node i el dashboard funcionin bé.

A2. Els tres nivells de QoS

- QoS 0, com a màxim un cop: s'envia i prou, sense confirmació. Pitjor cas: 0 vegades (es perd i ningú no se n'assabenta).
- QoS 1, almenys un cop: el receptor respon amb un PUBACK i, si no arriba, el missatge es torna a enviar. Pitjor cas: més d'una vegada (duplicat), per això qui el rep ha de suportar duplicats.
- QoS 2, exactament un cop: es fa servir un intercanvi de quatre missatges (PUBLISH, PUBREC, PUBREL i PUBCOMP). Pitjor cas: 1 vegada. És el més car.

El QoS és entre client i broker, no d'extrem a extrem: el subscriptor el rep amb el QoS més baix entre el de la publicació i el de la seva subscripció.

A3. Missatge retingut i diferència amb QoS 1

Un missatge retingut és un missatge que el broker es guarda: es queda l'últim de cada tema i l'envia de seguida a qualsevol client que s'hi subscrigui després, encara que sigui hores més tard.

El QoS 1 no guarda res per al futur: només assegura que el missatge arribi ara als que ja estan subscrits. El retingut és la memòria de l'últim valor; el QoS és la fiabilitat de l'enviament.

Exemple: l'estat del node (`online` / `offline`). Si es publica retingut, quan obrim el dashboard per primera vegada ja veiem si el node està connectat, sense esperar que torni a publicar.

A4. Last Will and Testament

En connectar-se, el client deixa al broker un missatge de "testament" (tema, contingut, QoS i retain), per exemple `.../status = offline`. Si el client desapareix sense acomiadar-se (es queda sense corrent o perd la Wi-Fi), el broker publica aquest missatge en nom seu. Si es desconnecta bé (amb DISCONNECT), no es publica.

Amb un keepalive de 60 s, el broker dona el client per mort quan passa 1,5 vegades el keepalive sense rebre res d'ell: fins a 90 s des de l'últim paquet. Si la connexió TCP es talla de manera que el broker ho nota, el testament surt abans.

A5. Sessió persistent

Si el client es connecta amb `clean_session` a fals, el broker li guarda la sessió quan es desconnecta: les seves subscripcions i els missatges de QoS 1 i 2 que li arribin mentre és fora. Quan es torna a connectar amb el mateix identificador de client, rep els missatges pendents i no cal que es torni a subscriure.

Els de QoS 0 no es guarden (a Mosquitto, per defecte): es perden. I si el client canvia d'identificador, el broker el tracta com un client nou i no li dona res del que tenia guardat.

A6. Barra inicial i comodí # enmig

Un tema sí que pot començar per /, però vol dir que el primer nivell és buit. `/uvic/se2627` i `uvic/se2627` són temes diferents, i un filtre com `uvic/#` no rep res de `/uvic/...`. Porta confusions i trenca els comodins, per això no ho farem.

El comodí # no es pot posar enmig: ha d'anar sol i al final (`uvic/se2627/pauvila/#`). Per a un nivell del mig es fa servir +, que substitueix un sol nivell: `uvic/se2627/pauvila/+/status`. Els comodins només serveixen per subscriure's, no per publicar.

Bloc B · Disseny del nostre sistema

B1. Espai de noms de temes

Seguim l'estructura del mòdul M7: del general al particular, sense accents, espais ni barra inicial. Les parts variables van entre < >.

Patró del tema	Per a què	Exemple de missatge
`uvic/se2627/pauvila/<node>/telemetria/<magnitud>`	Telemetria	`{"valor":23.4,"unitat":"C","ts":1793014800}`
`uvic/se2627/pauvila/<node>/status`	Estat de connexió (també és el Last Will)	`online / offline`
`uvic/se2627/pauvila/<node>/status/transmissio`	Estat de la transmissió	`running / paused`
`uvic/se2627/pauvila/<node>/cmd/<ordre>`	Ordres cap al node	`{"id":"o17","valor":"atura"}`
`uvic/se2627/pauvila/<node>/config/<parametre>`	Configuració del node	`{"id":"c05","parametre":"periode","valor":30}`
`uvic/se2627/pauvila/<node>/ack/<nom>`	Acusament del node	`{"id":"c05","ok":true,"valor":30}`
`uvic/se2627/pauvila/<node>/alarma/<magnitud>`	Alarma per llindar	`{"actiu":true,"valor":31.2,"ts":1793014800}`

<node> = node01, node02 · <magnitud> = temp, hum · <ordre> = transmissio · <parametre> = periode, llindars · <nom> = el nom de l'ordre o del paràmetre que es confirma. Hi hem afegit la configuració i les alarmes perquè la P3 les fa servir.

B2. L'identificador del node, abans o després del tipus de dada

Posem el node abans del tipus de dada: `uvic/se2627/pauvila/node01/telemetria/temp`, com al mòdul M7.

– Què és fàcil: tot el que és d'un node penja d'una sola branca. Amb `.../node01/#` es veu tot el node, el node es subscriu a les seves ordres amb una sola línia (`.../node01/cmd/#`) i, a la P3, l'ACL que deixa cada node només a la seva branca és una sola regla.

– Què és més difícil: veure la mateixa dada de tots els nodes. Cal el comodí + al mig (`uvic/se2627/pauvila/+/telemetria/temp`). Es pot fer, però és menys directe.

Amb el tipus primer (`.../telemetria/node01/temp`) passaria al revés: tota la telemetria seria una sola branca, però el que és d'un node quedaria repartit i els permisos per node serien més complicats.

B3. Format del missatge de telemetria (JSON)

Un missatge per magnitud, perquè la magnitud ja va al tema:

```
{"valor":23.4,"unitat":"C","ts":1793014800}
```

– `valor`: el número, amb un decimal.

– `unitat`: `C` per a la temperatura i `%` per a la humitat.

– `ts`: marca de temps en segons des de 1970 (temps Unix). El node agafa l'hora per NTP quan es connecta a la Wi-Fi. Si encara no la té, envia 0 i la posa el Node-RED quan rep el missatge.

No hi posem l'identificador del node: ja és al tema, i el Node-RED el pot treure de `msg.topic`. Repetir-lo serien més bytes a cada missatge i el risc que el tema i el JSON diguessin coses diferents.

B4. Temes retinguts

Tema	Retingut	Per què
`.../telemetria/<magnitud>`	No	Un valor vell es mostraria com si fos actual.
`.../status`	Sí	Qui obre el dashboard veu de seguida si el node és online. El Last Will també va retingut.
`.../status/transmissio`	Sí	Es veu si està aturat o en marxa sense esperar.
`.../cmd/<ordre>`	No	Una ordre vella no s'ha d'executar quan el node es torna a connectar.
`.../config/<parametre>`	Sí	En reiniciar-se, el node rep l'últim període i els últims llindars.
`.../ack/<nom>`	No	És la resposta a una ordre concreta i només té sentit en aquell moment.
`.../alarma/<magnitud>`	No	Són avisos puntuals. Retinguts, un dashboard nou mostraria alarmes velles.

B5. Trànsit amb dos nodes

Cada node publica 2 magnituds cada 10 s: 12 missatges per minut i node, 24 missatges per minut amb dos nodes.

Mida d'un missatge de temperatura amb QoS 0:

– capçalera fixa: 2 bytes

– longitud del tema: 2 bytes

– tema `uvic/se2627/pauvila/node01/telemetria/temp`: 42 bytes

– JSON `{"valor":23.4,"unitat":"C","ts":1793014800}`: 43 bytes

En total 89 bytes (el d'humitat, 88). Per minut: 12 × 89 + 12 × 88 = 2.124 bytes ≈ 2,1 kB/min, unes 127 kB per hora. Comptant també les capçaleres TCP/IP (uns 40 bytes per paquet) serien uns 3 kB/min. El broker

torna a enviar el mateix a cada subscriptor, i els missatges d'estat, ordres i acusaments són pocs. És molt poc trànsit.

Bloc C · El broker com a servei del sistema

C1. Paquet, servei i fitxers

- El paquet (`apt install mosquitto`) és el programa instal·lat al disc: l'executable `/usr/sbin/mosquitto`, la configuració per defecte i el fitxer que diu a systemd com engegar-lo. Que estigui instal·lat no vol dir que estigui funcionant.
- El servei és el programa en marxa, en segon pla. El gestiona systemd: l'engega, l'atura, el torna a engegar si cau i el pot arrencar amb el sistema.
- Els fitxers de configuració (`/etc/mosquitto/mosquitto.conf` i els de `/etc/mosquitto/conf.d/`) diuen com s'ha de comportar: ports, usuaris, persistència. Els de dades són on guarda el que ha de sobreviure a un reinici: `/var/lib/mosquitto/mosquitto.db` (retinguts i sessions). El registre va a `/var/log/mosquitto/`.

C2. Per què la configuració va a conf.d

El `mosquitto.conf` és del paquet. Si l'editem i després s'actualitza el paquet, apt ens preguntarà si el vol substituir i podem perdre els canvis. A més, aquest fitxer ja porta la línia `include_dir /etc/mosquitto/conf.d`, que carrega tots els `.conf` d'aquesta carpeta.

Així la nostra configuració queda en un fitxer nostre i separat: es veu de seguida què hem canviat, es pot guardar al repositori i, per tornar a l'original, n'hi ha prou amb treure'l.

C3. systemctl enable i systemctl start

- `systemctl start mosquitto` l'engega ara. No diu res de l'arrencada.
- `systemctl enable mosquitto` fa que s'engegui sol cada vegada que arrenca el Raspberry. No l'engega ara.

Si només fem start, funciona fins que reiniciem, i després del reinici ja no hi és. Si només fem enable, ara no funciona però després del reinici sí. Per això calen les dues. Al Raspberry Pi OS, apt ja el deixa activat i engegat en instal·lar-lo, però ho fem igualment per assegurar-ho.

C4. Listener a 0.0.0.0 o només a localhost

- A 0.0.0.0 (totes les interfícies): s'hi pot connectar qualsevol aparell de la xarxa del laboratori que arribi a la IP del Raspberry: el nostre node i el portàtil, però també qualsevol company. Sense usuari i contrasenya podria llegir-ho tot (subscrivint-se a #) o enviar ordres al node. Per això a la P3 hi posarem contrasenya i ACL.
- Només a localhost: només s'hi poden connectar els programes del mateix Raspberry, com el Node-RED. El node ESP32 i el portàtil no hi arribarien, així que no ens serviria.

Preparació pràctica

Eines al portàtil

Hem instal·lat MQTT Explorer 0.4.0-beta.6 i Mosquitto 2.1.2 per a Windows, que porta `mosquitto_pub` i `mosquitto_sub` (sense el servei del broker, que al portàtil no cal).

Ordres per instal·lar Mosquitto al Raspberry Pi (encara sense executar)

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo nano /etc/mosquitto/conf.d/p3.conf      # s'hi enganxa el fitxer de sota
sudo systemctl enable mosquitto              # que arrenqui sol amb el Raspberry
sudo systemctl restart mosquitto             # engegar-lo ara amb la configuració nova
systemctl status mosquitto                   # ha de dir enabled i active (running)
```

Fem `restart` en lloc de `start` perquè apt ja l'engega en instal·lar-lo, i així carrega el fitxer nou. El Node-RED continua al port 1880 i Mosquitto és al 1883, així que no es molesten.

Fitxer de configuració mínim: /etc/mosquitto/conf.d/p3.conf

```
listener 1883
allow_anonymous true
persistence true
```

`listener 1883` escolta al port 1883 a totes les interfícies. `allow_anonymous true` cal perquè, a Mosquitto 2, si es defineix un listener sense usuaris es rebutgen tots els clients; a la P3 ho canviarem per contrasenya i ACL. `persistence true` guarda els retinguts i les sessions al disc. Els dos fitxers són al repositori, a la carpeta `docs/pre3/`.

Prova contra un broker públic

El 26/10, test.mosquitto.org no responia al port 1883 (temps d'espera esgotat des del portàtil, a la Wi-Fi del laboratori i a una altra xarxa, i també des del Raspberry), tot i que la seva web sí que s'obria. Amb la mateixa xarxa, broker.hivemq.com, un altre broker públic de proves sense autenticació, sí que funcionava, i hem fet la prova amb aquest. Hem fet servir el prefix `uvic/se2627/pauvila/` i dades inventades. Des del cmd:

```
"C:\Program Files\mosquitto\mosquitto_sub" -h broker.hivemq.com -t "uvic/se2627/pauvila/#" -v
"C:\Program Files\mosquitto\mosquitto_pub" -h broker.hivemq.com -t
"uvic/se2627/pauvila/node01/telemetria/temp"
    -m "{\"valor\":23.4,\"unitat\":\"C\",\"ts\":1793014800}"
```

El missatge arriba a `mosquitto_sub` (figura 1) i també a MQTT Explorer, connectat al mateix broker i subscrit només a `uvic/se2627/pauvila/#` (figura 2).

Al Node-RED del Raspberry, un node `inject` envia la mateixa temperatura a un `mqtt out` que la publica al tema `.../node01/telemetria/temp`, i un `mqtt in` subscrit a `uvic/se2627/pauvila/#` la rep i la mostra a la pestanya de depuració (figura 3).



Figura 1. A dalt, mosquitto_pub publica una temperatura inventada; a baix, mosquitto_sub la rep.

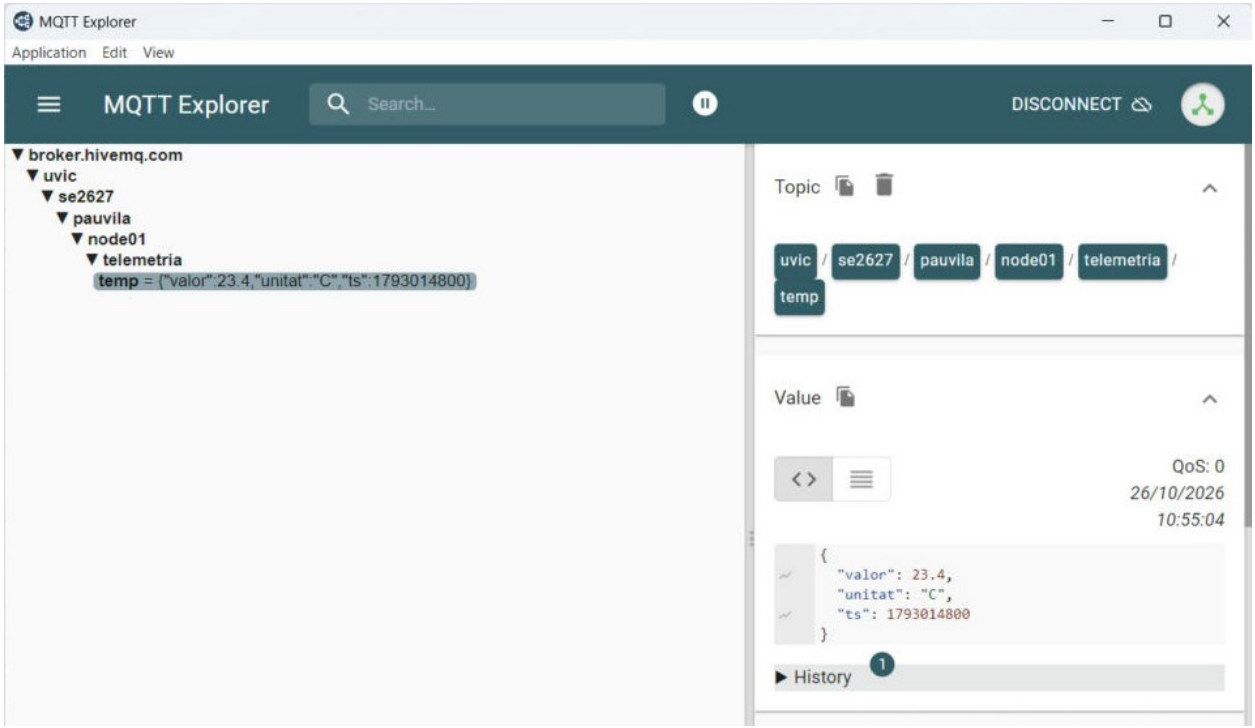


Figura 2. MQTT Explorer mostra l'arbre de temes uvic/se2627/pauvila/... i el JSON rebut.

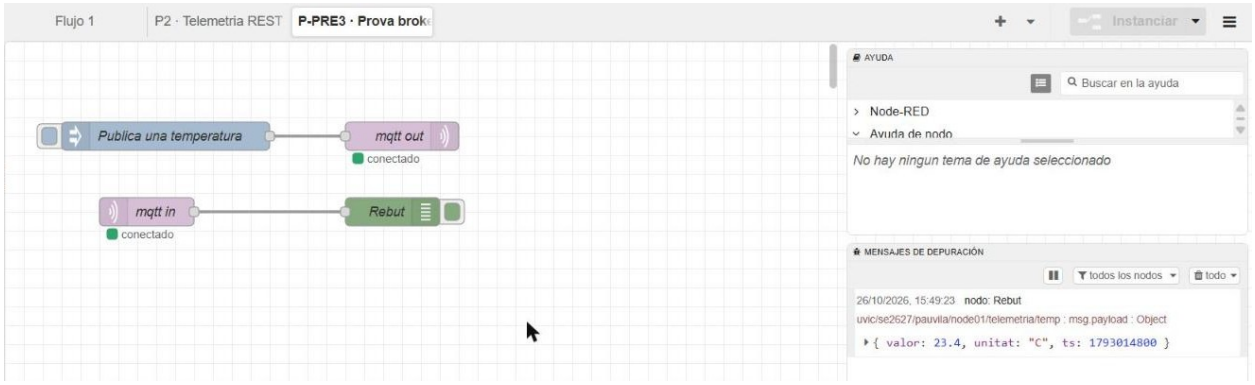


Figura 3. Node-RED del Raspberry: el mqtt out publica, el mqtt in ho rep i surt a la depuració.